

VisionInspect AI

AI-Powered Manufacturing Defect Detection & Quality Inspection System

Milestone 4

Deployment & Containerization Documentation

[Docker](#) · [Docker Compose](#) · [Cloudflare Tunnel](#) · [FastAPI](#) · [React \(Vite\)](#) · [PostgreSQL](#)

Prepared for Final Year Project Documentation, Internship Submission,
Project Review & Mentor Presentation

Akshaya Reddy

[August 2026](#)

Table of Contents

1. Deployment Overview
2. Backend Container (FastAPI + AI Engine)
3. Frontend Container (React + Vite)
4. Orchestration with Docker Compose
5. Persistent Volumes
6. Secure Public Exposure (Cloudflare Tunnel)
7. Automated Fallback Script (start_app.py)
8. Deployment Advantages

1. Deployment Overview

VisionInspect AI is packaged and shipped as a set of isolated Docker containers, orchestrated together with Docker Compose and exposed to the internet through a Cloudflare Tunnel. This section documents the containerization strategy for both the backend and frontend services, the orchestration layer that ties them together, the persistent storage used to protect data across restarts, the secure public-exposure mechanism, and the automated fallback deployment script used when Docker or Cloudflare are unavailable.

Custom Dockerfiles were created for both the frontend and backend, allowing each service to be built, versioned, and deployed independently while still communicating over a shared, private Docker network.

2. Backend Container (FastAPI + AI Engine)

2.1 Base Image

The backend image is built on **python:3.10-slim**, chosen for a small footprint while remaining fully compatible with the FastAPI, SQLAlchemy, and PyTorch/OpenCV dependency chain.

2.2 System-Level Configuration

- Installs **libgl1** and **libglib2.0-0**, the system-level shared libraries required by OpenCV for image decoding and processing inside a headless container (these are not present in the slim base image by default).

2.3 AI Model Packaging

- The **results/** directory, containing the trained PatchCore anomaly-detection weights (**.ckpt** files), is copied directly into the Docker image at build time.
- This bakes the trained model into the production image itself, so the container never needs to reach an external model registry or storage bucket on boot — inference is available immediately after the container starts.

2.4 Execution

- The FastAPI application is served with **uvicorn** on port **8000**, the same port other services in the Compose network use to reach the API.

```
FROM python:3.10-slim

RUN apt-get update && apt-get install -y \
    libgl1 libglib2.0-0 && \
    rm -rf /var/lib/apt/lists/*

WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
```

```
COPY . .
COPY results/ ./results/

EXPOSE 8000
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Figure 1: Representative backend Dockerfile structure.

3. Frontend Container (React + Vite)

3.1 Multi-Stage Build

The frontend uses a two-stage Docker build to keep the final production image as small as possible — the Node.js toolchain used to compile the app never ships in the final image.

3.2 Stage 1 — Builder

- Uses **node:18-alpine** to install npm dependencies and run the Vite build, compiling the React application into static HTML, CSS, and JS assets.

3.3 Stage 2 — Production

- Uses a lightweight **nginx:alpine** image to serve only the compiled static assets from Stage 1 on port **80**.
- Nginx is configured to route all Single Page Application (SPA) requests back to **index.html**, so client-side React Router paths resolve correctly on a hard refresh.

```
# Stage 1: Builder
FROM node:18-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
ARG VITE_API_BASE_URL
ENV VITE_API_BASE_URL=$VITE_API_BASE_URL
RUN npm run build

# Stage 2: Production
FROM nginx:alpine
COPY --from=builder /app/dist /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

Figure 2: Representative frontend multi-stage Dockerfile.

4. Orchestration with Docker Compose

A master **docker-compose.yml** file orchestrates the entire infrastructure on a private, isolated bridge network named **vision_network**. This keeps inter-service traffic (e.g. backend-to-database queries) off the public internet entirely, while still allowing controlled external access through the Cloudflare Tunnel service.

4.1 Services Managed by the Orchestrator

Service	Image / Build	Responsibility
db	postgres:15-alpine	Runs the PostgreSQL database that persists all operational data for the platform.
backend	Custom build (FastAPI)	Builds the FastAPI + AI engine container and connects it to the database service.
frontend	Custom build (Nginx/React)	Builds the compiled React app and serves it via Nginx; injects VITE_API_BASE_URL a
cloudflared	cloudflare/cloudflared	Daemon that connects the internal Docker network to Cloudflare's edge network for se

4.2 Build-Time Environment Injection

The frontend service dynamically injects the **VITE_API_BASE_URL** environment variable at build time (rather than at runtime), ensuring every compiled API call in the static bundle correctly targets the backend's public Cloudflare subdomain instead of a local address.

```
networks:
  vision_network:
    driver: bridge

services:
  db:
    image: postgres:15-alpine
    networks: [vision_network]
    volumes:
      - postgres_data:/var/lib/postgresql/data

  backend:
    build: ./backend
    depends_on: [db]
    networks: [vision_network]
    volumes:
      - uploads_data:/app/uploads
    ports: ["8000:8000"]

  frontend:
    build:
      context: ./frontend
    args:
      VITE_API_BASE_URL: ${VITE_API_BASE_URL}
    networks: [vision_network]
    ports: ["80:80"]
```

```
cloudflared:  
  image: cloudflare/cloudflared:latest  
  command: tunnel run  
  networks: [vision_network]
```

Figure 3: Representative docker-compose.yml structure.

5. Persistent Volumes

To prevent data loss whenever a container is restarted, rebuilt, or destroyed, two named Docker Volumes are mounted into the stack.

Volume	Mounted To	Purpose
postgres_data	db service (/var/lib/postgresql/data)	Ensures all user accounts and historical inspection records persist across
uploads_data	backend service (/app/uploads)	Shared volume that permanently stores all uploaded product images and

Because these volumes live outside any single container's writable layer, destroying and recreating a container (for example, during a redeploy) never wipes the database or the uploaded image history.

6. Secure Public Exposure (Cloudflare Tunnel)

To make the application reachable from the internet without opening inbound firewall ports or exposing the host server's IP address, VisionInspect AI integrates Cloudflare Tunnels via the **cloudflared** daemon.

6.1 How It Works

- Instead of configuring inbound port-forwarding, **cloudflared** establishes a secure, **outbound-only** QUIC connection from inside the Docker network to Cloudflare's global edge network.
- Because the connection is outbound-only, no inbound firewall rule ever needs to be opened on the host machine, which removes an entire class of direct-attack and port-scanning exposure.
- External users reach the React frontend and FastAPI backend through encrypted HTTPS subdomains issued by Cloudflare, while the host machine's true IP address and internal network topology remain hidden.

6.2 Security Benefit

This architecture shields the host from direct-connection DDoS attacks and IP-based reconnaissance, since every request is terminated and filtered at Cloudflare's edge before ever reaching the tunnel.

7. Automated Fallback Script (start_app.py)

To account for environments where Docker or Cloudflare Quick Tunnels experience outages or permission restrictions, a custom Python automation script, **start_app.py**, was built as a robust deployment fallback that runs the stack outside of Docker entirely.

7.1 Responsibilities

- **Port cleanup:** identifies and terminates any zombie processes blocking the required ports (**5173** for the Vite dev server and **8000** for the FastAPI backend) before startup.
- **Dependency recovery:** forces the installation of OpenCV dependencies and bypasses restrictive SSL / PyPI proxy errors that can block installation in locked-down network environments.
- **Asynchronous launch:** starts the backend and frontend processes concurrently rather than sequentially, minimizing total startup time.
- **Public access fallback:** initiates a **localtunnel** connection as an alternative to Cloudflare, giving immediate public internet access to the running application when the primary tunnel path is unavailable.

```
if __name__ == "__main__":
    kill_zombie_processes(ports=[5173, 8000])
    ensure_opencv_dependencies(force=True, bypass_ssl_errors=True)

    start_process_async("backend", cmd="uvicorn main:app --port 8000")
    start_process_async("frontend", cmd="npm run dev -- --port 5173")

    start_localtunnel(port=8000, subdomain="visioninspect-api")
    start_localtunnel(port=5173, subdomain="visioninspect-app")
```

Figure 4: Simplified control flow of start_app.py.

8. Deployment Advantages

- **Isolation:** each service (database, backend, frontend, tunnel) runs in its own container with its own dependency set, preventing version conflicts between the Python and Node.js toolchains.
- **Reproducibility:** a fresh environment can be brought up identically on any machine with a single **docker compose up** command.
- **Data safety:** named volumes decouple application data from container lifecycle, so red deploys and rebuilds never risk data loss.
- **Security by default:** the Cloudflare Tunnel's outbound-only model removes the need for inbound port-forwarding and hides the host's real network identity.
- **Resilience:** the start_app.py fallback script ensures the application can still be brought online and reached publicly even if Docker or the primary tunnel provider is unavailable.